

	UNIVERSIDAD DE SANTIAGO DE CHILE FACULTAD DE INGENIERÍA DEPARTAMENTO DE INGENIERÍA INFORMÁTICA	 UdeSantiago de Chile
--	---	--

TALLER DE BASE DE DATOS DIURNO 1-2026
Laboratorio 2

Ayudante: Fernando Solís	Profesor: Matías Calderón
---------------------------------	----------------------------------

Enunciado Laboratorio 2

Entrega: 15 de junio de 2026 (hasta las 12:00 hrs)

Implementación de Sistemas de Información con Arquitectura Multi-Capa, Bases de Datos Avanzadas y Seguridad JWT

1. Identificación del Curso

- **Asignatura:** Programación de Aplicaciones / Bases de Datos Avanzada
- **Nivel:** Pregrado / Ingeniería
- **Unidad Temática:** Integración de API REST, SQL Avanzado y Seguridad.

2. Actualización de requerimientos técnicos

Para esta versión del laboratorio, se deben considerar los siguientes ajustes:

- Motor de Base de Datos: PostgreSQL con la extensión PostGIS habilitada.
- Indexación Espacial: Es obligatorio el uso de índices GIST para columnas espaciales para optimizar consultas de proximidad y contención.
- Capa de Servicios: La API debe ser capaz de procesar y retornar formatos geoespaciales como GeoJSON.

3. Objetivos de Aprendizaje

1. Diseñar esquemas de bases de datos relacionales normalizados, con uso de datos geoespaciales.
2. Implementar lógica de servidor persistente mediante procedimientos almacenados y disparadores.
3. Desarrollar una capa de servicios (API) que garantice la integridad y seguridad de los datos.
4. Gestionar la autenticación y autorización de usuarios mediante el estándar JSON Web Tokens (JWT).
5. Documentar técnicamente procesos de despliegue y consumo de servicios.

4. Requerimientos Técnicos (Arquitectura)

- **Motor de Base de Datos:** PostgreSQL.
- **Backend:** Desarrollo de una API REST utilizando lenguajes de programación de propósito general (ej. Python, JavaScript/TypeScript, Go, Java, C#). Independiente del lenguaje **no se permite el uso de ORMs. (En consecuencia, Django queda descartado)**
- **Frontend:** Interfaz web funcional que consuma los recursos de la API (React, Vue, Angular o Vanilla JS).
- **Seguridad:** Implementación obligatoria de middleware de autenticación. Las rutas deben estar protegidas según el rol del usuario (RBAC).

A continuación, se presentan los 5 dominios. Cada grupo debe modelar la base de datos y construir los endpoints de la API para resolver las siguientes 6 tareas específicas.

Grupo 1: Sistema de Gestión de Torneos de Tiro con Arco

Contexto: Una plataforma para gestionar arqueros, categorías (ej. Recurvo, Compuesto), torneos y puntuaciones por ronda.

- **Componente Espacial:** Ubicación exacta de los blancos y de los arqueros en la línea de tiro.
- **Tarea PostGIS:** Implementar un **Trigger** que valide que la posición del arquero se encuentre dentro de la "Zona de Competencia" (un polígono definido en la base de datos) antes de registrar una puntuación.
- **Procedimiento Almacenado:** Calcular la distancia real entre la posición del arquero (POINT) y el centro del blanco asignado para verificar el cumplimiento de las normativas de distancia de la categoría.
- **Análisis Espacial de Desempeño:** ¿Cómo implementaría una consulta que determine si existe una correlación entre la ubicación del blanco en el terreno (coordenadas X, Y) y la precisión del arquero, considerando posibles factores ambientales como zonas de viento registradas como polígonos? (Ej. Viento, lluvia, frío, etc.)
- **Validación de Seguridad:** Diseñe un Trigger que utilice ST_Distance. Si un arquero registra un disparo, pero su ubicación GPS está a menos de 5 metros de otro arquero activo, ¿cómo bloquearía la inserción por motivos de seguridad?
- **Geocercado:** El enunciado exige validar que el torneo esté activo. Agregue una restricción: el puntaje solo es válido si el POINT del arquero está contenido (ST_Contains) dentro del polígono oficial del campo de tiro.

Grupo 2: Sistema de Administración Académica Universitaria

Contexto: Una intranet para gestionar estudiantes, asignaturas, requisitos, semestres y calificaciones.

- **Componente Espacial:** Geometría de los edificios del campus y ubicación de las salas.
- **Tarea PostGIS:** Un **Endpoint** de la API que reciba la ubicación GPS del estudiante y devuelva la sala más cercana donde tiene clases en ese momento.
- **Vista Materializada:** Un mapa de calor de "Densidad Estudiantil" por edificio, basado en la cantidad de alumnos inscritos en cursos asignados a polígonos de infraestructura.
- **Optimización de Traslados:** Para el proceso de inscripción, ¿cómo generaría un reporte que sugiera al estudiante secciones de cursos cuyos edificios estén a una distancia caminable (ej. < 300m) de su ubicación actual o de su clase anterior?
- **Accesibilidad:** Utilizando las geometrías de los edificios del campus, ¿cómo identificaría qué salas de clase son accesibles para estudiantes con movilidad reducida basándose en la cercanía a rampas (puntos de interés) registradas en PostGIS?
- **Zonificación:** Cree una Vista Materializada que agrupe la tasa de reprobación no solo por asignatura, sino por "Distrito de Vivienda" de los estudiantes para identificar zonas geográficas con bajo rendimiento académico.

Grupo 3: Gestor de Clanes y Raids para MMORPG

Contexto: Una aplicación para gestionar jugadores, sus personajes, eventos (Raids), asistencia y distribución de botín (Loot).

- **Componente Espacial:** Ubicación de los Clanes y ubicación de las Raids.
- **Tarea PostGIS:** Un **Endpoint** de la API que reciba la ubicación GPS del usuario y devuelva los clanes más cercanos a su ubicación.
- **Vista Materializada:** Un mapa de calor de "Clanes mejor rankeados".
- **Mecánicas de Proximidad:** En un evento de Raid, ¿cómo programaría un Procedimiento Almacenado que distribuya el botín (Loot) solo a los personajes cuya ubicación espacial (GEOMETRY) esté dentro de un radio de 50 unidades del punto de muerte del jefe (Boss)?
- **Formación de Grupos:** El enunciado pide buscar por clase de personaje. Agregue: ¿Cómo filtraría a los "Healers" disponibles que se encuentren en la misma "Región del Mapa" que el "Tanque" líder?
- **Auditoría Territorial:** Al transferir el liderazgo del clan, registre las coordenadas del mundo virtual donde se realizó el acto para crear un mapa histórico de "Sedes de Poder" del clan.

Grupo 4: Plataforma de E-Commerce B2B

Contexto: Una tienda mayorista que maneja inventario de productos, clientes de empresas, carritos de compras y facturación.

- **Componente Espacial:** Ubicación de almacenes y direcciones de entrega de clientes.
- **Tarea PostGIS:** Un **Procedimiento Almacenado** que, al procesar el *Checkout*, asigne automáticamente el pedido al almacén más cercano utilizando la función ST_Distance.
- **Trigger:** Impedir la creación de una orden si la dirección de entrega del cliente se encuentra fuera del área de cobertura (polígono de servicio) de la empresa.
- **Logística de Última Milla:** Al procesar el Checkout, ¿cómo calcularía cuál de los almacenes (puntos) tiene el stock necesario y está geográficamente más cerca de la empresa cliente para minimizar costos de envío?
- **Zonas de Exclusión:** Implemente un Trigger que impida la venta de categorías específicas (ej. químicos peligrosos) si la ubicación del cliente se encuentra dentro de un polígono de "Zona Residencial Protegida".
- **Análisis de Mercado:** Utilice una Vista Materializada para mostrar el volumen de ventas proyectado sobre un mapa, agrupando pedidos por comunas o distritos postales utilizando ST_Union.

Grupo 5: Panel de Control de Infraestructura Cloud

Contexto: Un sistema para que los usuarios desplieguen y gestionen instancias de servidores virtuales, bases de datos y consumo de ancho de banda.

- **Componente Espacial:** Coordenadas de los Datacenters y regiones de servicio.
- **Tarea PostGIS:** Una consulta que devuelva la latencia proyectada basada en la distancia esférica entre la ubicación del usuario (IP geolocalizada) y el nodo de infraestructura más próximo.
- **Índices:** Aplicación de índices **GIST** sobre las coordenadas de los servidores virtuales para consultas de disponibilidad por región geográfica.
- **Soberanía de Datos:** Cree un Trigger que bloquee la creación de un servidor si el tipo de dato que el usuario pretende alojar tiene restricciones legales que le impidan salir de un polígono geográfico específico (ej. El usuario intenta crear un servidor con una base de datos alojada en otra región geográfica que el propio país por legislación no permite).
- **Resiliencia Geográfica:** ¿Cómo diseñaría un Endpoint de la API que reciba el ID de un servidor y devuelva los 3 Datacenters más cercanos que no pertenezcan a la misma placa tectónica o zona de riesgo de desastres?
- **Facturación por Distancia:** Si el sistema cobra por ancho de banda, ¿cómo integraría la distancia entre el servidor de origen y el punto de acceso del usuario para aplicar un recargo por latencia geográfica?

5. Entregables y Normas de Envío

Para que el proyecto sea considerado como entregado y válido para evaluación, cada grupo deberá proporcionar un repositorio de **GitHub** (o GitLab/Bitbucket) que contenga la siguiente estructura y elementos:

1. **Código Fuente:** Organizado en carpetas claras para backend y frontend.
2. **Script de Base de Datos:** Archivo .sql con la definición de tablas, relaciones, triggers, procedimientos, vistas materializadas e inserción de datos de prueba (seeders).
3. **README.md (Documentación Principal):** Este archivo es el corazón de la entrega. Debe incluir:
 - Arquitectura utilizada y tecnologías.
 - **Manual de Instalación y Despliegue:** Instrucciones paso a paso para ejecutar el proyecto en un entorno local (ej. configuración de variables de entorno, comandos para levantar contenedores Docker o instalar dependencias).
 - Documentación de la API (endpoints principales y ejemplos de JSON).
4. **Carpeta Presentación:** Debe incluir las diapositivas (PDF o PPTX) que se utilizarán para la defensa del proyecto.

Regla Crítica de Evaluación (Criterio de Aceptación)

Funcionabilidad del Manual de Instalación: Es responsabilidad estricta de los alumnos asegurar que el sistema pueda ser desplegado siguiendo exclusivamente las instrucciones detalladas en el archivo README.md.

Si al momento de seguir el manual de instalación el sistema no funciona, no compila o presenta errores de configuración insalvables, el proyecto será calificado automáticamente con la nota mínima. Se recomienda probar el proceso de instalación desde cero en una máquina limpia antes de la entrega final.

Método de evaluación: La nota final de cada laboratorio corresponderá al promedio simple entre la nota de presentación y la nota de código. Sin embargo, para que la nota de presentación sea considerada, **la nota de código debe ser igual o superior a 4,0**. En caso contrario, la nota final corresponderá únicamente a la nota de código, resultando en reprobación de ese laboratorio.